

AutoFishingRig

An automated fishing rig — a controller + sensor + reel that detects a fish bite via line-tension spike and reels the fish in. Manually cast; automatic from tension-setup through landing.

Version **0.1**

Dictionary last updated **2026-05-22**

Generated **2026-05-23**

Validation:  **valid**

Modernization: **ADRs: 1 · Budgets/TPMs: 2/2 · SLOs: 1 · Bounded contexts: 0**

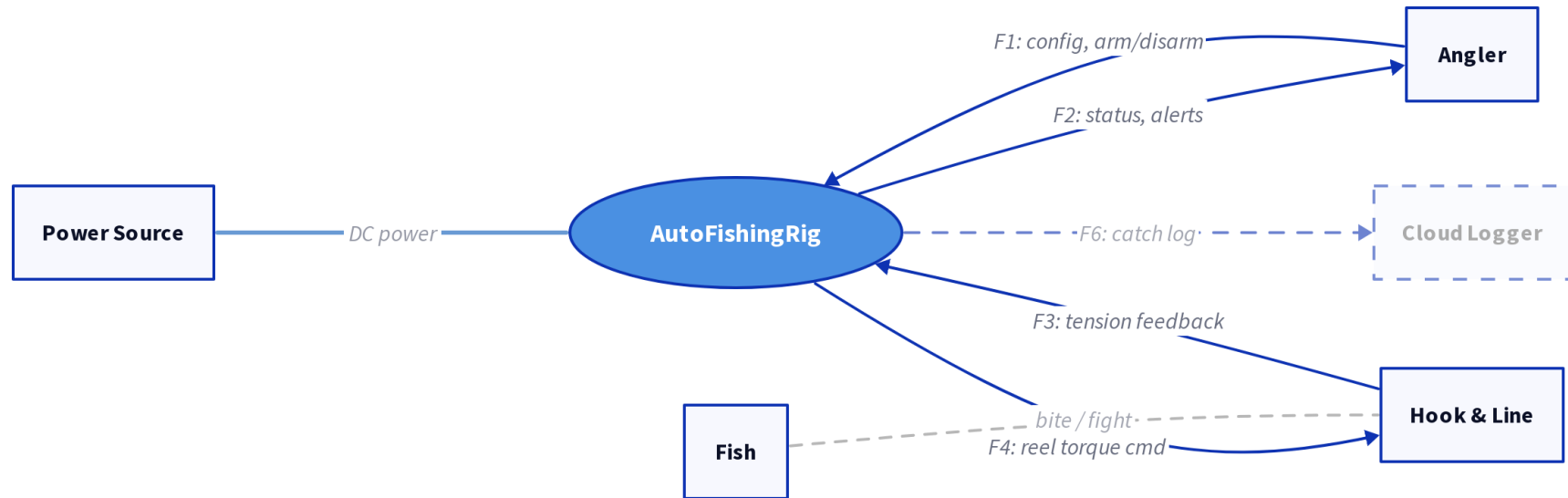
Contents

Stage 1 — Context Diagram	3
Context Diagram	4
Stage 2 — Level-1 DFD	5
Level-1 DFD	6
Stage 3 — CSPECs	7
Bite Detector	8
Stage 4 — PSPECs	9
Acquire Tension	10
Reel Controller	12
Serve UI	13
Cloud Forward	14
Stage 5 — Architecture	15
AFD (Flow Diagram)	16
AID (Interconnect Diagram)	17
Modules	18
Main Controller Board	19
Angler Mobile App	21
Interconnects	22
BLE Link	23
Modernization	25
ADRs	26
BLE chosen for controller ↔ app transport	27
SLOs Summary	29
Runbooks	30
slo bite to set burn	31
tension sensor stuck	32

Stage 1 — Context Diagram

5 terminator(s)

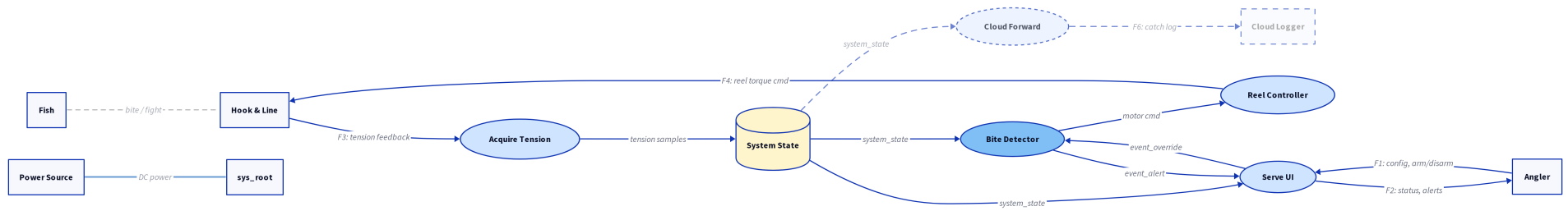
Context Diagram



Stage 2 — Level-1 DFD

5 process(es)

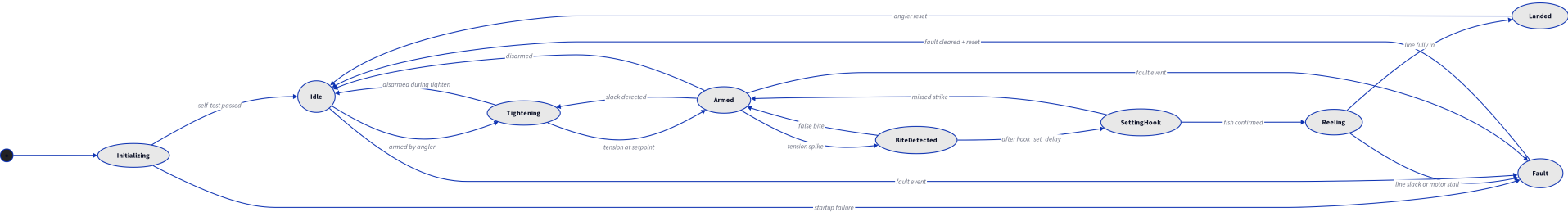
Level-1 DFD



Stage 3 — CSPECs

1 CSPEC(s)

Bite Detector



Stage 4 — PSPECs

4 PSPEC(s)

PSPEC — Acquire Tension

Process: [proc_acquire_tension](#) (level-1 DFD) *Generated from `dictionary.yaml`. Do not hand-edit.*

INPUTS

Flow	From	Medium
<code>flow_f3_tension</code> — F3: tension feedback	<code>term_line</code>	analog sensor → ADC

OUTPUTS

Flow	To
<code>flow_tension_to_state</code> — tension samples	<code>store_system_state</code>

TRANSFORMATION (textual)

```
Every sampling cycle:
  Read F3 TENSION from the analog input.
  Convert to engineering units using the sensor calibration.
  Update TENSION SAMPLES in store_system_state with the latest value
  and append it to the recent-history buffer (retain N most recent).
```

COMPUTATIONAL CONSTRAINTS

- **Frequency:** 50–200 Hz sampling rate (configurable; default 100 Hz)
- **Accuracy:** $\pm 1\%$ of measured tension
- **Timing:** Sample → `store_system_state` write latency < 5 ms

VERIFICATION

- **Methods:** test, analysis
- **Coverage target:** 95.0%
- **Validation scenarios:**
 - Sustained 200 Hz sampling for 10 minutes without buffer overflow
 - Calibration drift < 0.5% over 24-hour soak test
 - Recovery from transient ADC fault within one sample period

OBSERVABILITY

Metrics:

- `tension_samples_total` *counter* — Total tension samples acquired
- `tension_newtons` *gauge* (N) — Latest tension reading in Newtons

- `tension_adc_read_seconds` *histogram* (s) — ADC read latency

Traces:

- `tension.acquire_cycle` — One full sample-and-write cycle

Log categories:

- `tension.calibration` (*level: info*)

Alerts:

- `tension_sensor_stuck` (*warning*) — when `rate(tension_samples_total[1m]) == 0` → [runbook](#)

COMMENTS

First-cut PSPEC. Recent-history buffer size N pending Stage 5 sizing — see TSPEC notes once that artifact exists.

Not a formal part of the specification (1988 §13.5).

Format: 2000 Fig 4.46 — INPUTS / OUTPUTS / TRANSFORMATION. See [../../../../toolkit/PSPEC_DESIGN.md](#).

PSPEC — Reel Controller

Process: [proc_reel_controller](#) (level-1 DFD) *Generated from dictionary.yaml. Do not hand-edit.*

INPUTS

Flow	From	Medium
flow_motor_command — motor cmd	proc_bite_detector	—

OUTPUTS

Flow	To
flow_f4_torque — F4: reel torque cmd	term_line

TRANSFORMATION (textual)

```
On receipt of MOTOR CMD:
  Decode direction, target torque, and speed.
  Enforce safety limits – max current, stall threshold, and max
  continuous run time.
  Drive the reel actuator via F4 REEL TORQUE CMD with the bounded
  torque value at the requested direction.
  When stall or current limit is reached, hold torque at the
  enforced limit until MOTOR CMD changes.
```

COMPUTATIONAL CONSTRAINTS

- **Accuracy:** Commanded torque honored to within $\pm 5\%$ under non-saturated load
- **Timing:** MOTOR CMD $\rightarrow$ F4 REEL TORQUE CMD latency < 10 ms

COMMENTS

Safety limits live in this PSPEC; the bite-detection state machine (CSPEC for Bite Detector) trusts that they are enforced here.

Not a formal part of the specification (1988 §13.5).

Format: 2000 Fig 4.46 — INPUTS / OUTPUTS / TRANSFORMATION. See [../../../../toolkit/PSPEC_DESIGN.md](#).

PSPEC — Serve UI

Process: [proc_serve_ui](#) (level-1 DFD) *Generated from dictionary.yaml . Do not hand-edit.*

INPUTS

Flow	From	Medium
flow_f1_angler_config — F1: config, arm/disarm	term_angler	UI
flow_state_to_ui — system_state	store_system_state	—
flow_alert — event_alert	proc_bite_detector	—

OUTPUTS

Flow	To
flow_f2_status — F2: status, alerts	term_angler
flow_override — event_override	proc_bite_detector

TRANSFORMATION (textual)

Render F2 STATUS ALERTS from the current SYSTEM STATE (tension samples + bite-detection mode + reel state) and from any EVENT ALERT received.
 Display F1 CONFIG ARM DISARM controls to the angler.
 On angler override actions taken in the UI (manual cast/reel, disarm), emit EVENT OVERRIDE.

COMPUTATIONAL CONSTRAINTS

- **Frequency:** UI refresh ≥ 10 Hz; alert latency < 100 ms

COMMENTS

Presentation layer. Decisions about which alerts to surface vs suppress live with the user; this PSPEC just renders what arrives.

Not a formal part of the specification (1988 §13.5).

Format: 2000 Fig 4.46 — INPUTS / OUTPUTS / TRANSFORMATION. See [../.../toolkit/PSPEC_DESIGN.md](#) .

PSPEC — Cloud Forward

Process: [proc_cloud_forward](#) (level-1 DFD) *Generated from `dictionary.yaml`. Do not hand-edit.*

INPUTS

Flow	From	Medium
<code>flow_state_to_cloud</code> — <code>system_state</code>	<code>store_system_state</code>	—

OUTPUTS

Flow	To
<code>flow_f6_catch_log</code> — F6: catch log	<code>term_cloud</code>

TRANSFORMATION (textual)

```
If cloud forwarding is enabled by configuration:
  When SYSTEM STATE indicates a completed catch or session event,
  issue F6 CATCH LOG containing the relevant fields (timestamp,
  tension peak, bite-to-set delay, reel-in duration).
Otherwise, do nothing.
```

COMPUTATIONAL CONSTRAINTS

- **Timing:** Best-effort; not in the bite-detection critical path

COMMENTS

Optional bubble. The "issue" keyword in the body marks F6 CATCH LOG as time-transient (1988 §13.3) — emitted once per event, not held.

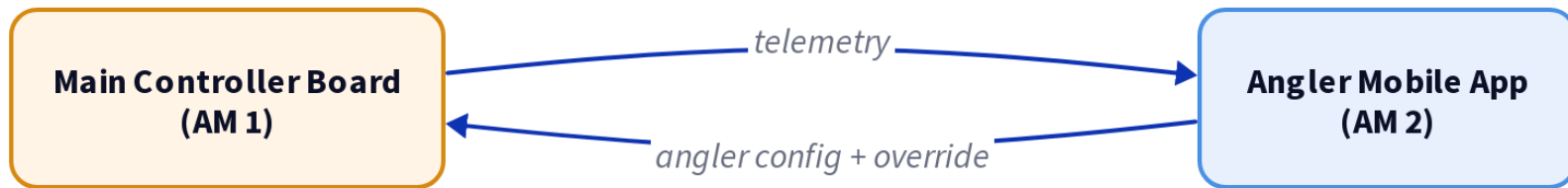
Not a formal part of the specification (1988 §13.5).

Format: 2000 Fig 4.46 — INPUTS / OUTPUTS / TRANSFORMATION. See [../.../toolkit/PSPEC_DESIGN.md](#).

Stage 5 — Architecture

2 module(s), 1 interconnect(s)

AFD (Flow Diagram)



AID (Interconnect Diagram)



Modules

AMS — Main Controller Board (AM 1)

Module: [am_controller](#) *Generated from `dictionary.yaml`. Do not hand-edit.*

DESCRIPTION

ESP32-WROOM-32 module on a custom PCB. Onboard: 16-bit ADC for the tension load cell, H-bridge motor driver for the reel, BLE antenna, optional WiFi for cloud. Firmware is FreeRTOS + the HP toolkit-derived CSPEC/PSPEC implementations.

CROSS-REFERENCE (allocation)

Requirements component	Kind
<code>proc_acquire_tension</code>	process
<code>proc_reel_controller</code>	process
<code>proc_cloud_forward</code>	process
<code>proc_bite_detector</code>	process (state-rich; CSPEC lives here)
<code>store_system_state</code>	data store

DESIGN RATIONALE

Chose ESP32 over a Cortex-M0+ for BLE + WiFi in a single package, simplifying both BoM and firmware. Onboard ADC avoids an external I2C ADC.

DESIGN JUSTIFICATION

MIPS budget: 200 Hz tension sampling + 20 Hz BLE notify + motor PWM is < 5% of ESP32 capacity at 80 MHz. RAM: state machine + recent-history buffer < 16 KB.

REQUIRED CONSTRAINTS

- **Reliability:** MTBF > 5000 hours of field operation.
- **Safety:** Motor cutoff on stall current (>2× nominal) within 50 ms.
- **Physical:** Fits in a 100×60×25 mm splashproof enclosure.
- **Cost:** BOM ≤ \$45 in 100-unit lots.

INTERFACES

Inputs: F3 TENSION (analog, 0–3.3V from load-cell amplifier); reel encoder feedback (quadrature, GPIO). Outputs: F4 REEL TORQUE CMD (PWM H-bridge); BLE GATT notifications (telemetry, alerts). WiFi/HTTPS (optional cloud forwarding).

BUDGETS (allocations to this module)

Budget	Unit	This module	System target	Reserve
<code>budget_bite_to_set_latency</code> — Bite detection → hook-set latency (end-to-end)	ms	170.0	200.0	30.0
<code>budget_telemetry_to_app_latency</code> — Telemetry → app render latency (informational; not safety-critical)	ms	100.0	300.0	50.0

TPMs (tracking this module's budgets)

TPM	Unit	Current	Growth allowance	Threshold
<code>tpm_bite_to_set_currently</code> — Measured bite-to-set latency (lab bench)	ms	165.0	35.0	200.0

SLOs (apply to this module)

SLO	Target	Window	Error budget
<code>slo_bite_to_set_latency</code> — Bite detection → hook-set latency	0.2 seconds	7d	1.0%

Format: 2000 §4.2.5.4 — typical AMS contents. See [../ARCH_DESIGN.md](#).

AMS — Angler Mobile App (AM 2)

Module: [am_angler_app](#) Generated from `dictionary.yaml` . Do not hand-edit.

DESCRIPTION

Cross-platform mobile app (Flutter). Pairs with the controller via BLE; presents a single-screen dashboard with live tension, mode indicator, recent-events list, and config controls.

CROSS-REFERENCE (allocation)

Requirements component	Kind
<code>proc_serve_ui</code>	process

DESIGN RATIONALE

Flutter chosen to ship one codebase to iOS and Android. BLE is the only required transport for first release; WiFi/cloud is out-of-scope for the app.

DESIGN JUSTIFICATION

UI refresh ≥ 10 Hz is trivial at the app's compute budget. BLE notifications drive the display directly with no polling.

REQUIRED CONSTRAINTS

- **Reliability:** Reconnects to controller within 5 s after BLE loss.
- **Physical:** Runs on iOS ≥ 15 , Android ≥ 11 .

BUDGETS (allocations to this module)

Budget	Unit	This module	System target	Reserve
<code>budget_telemetry_to_app_latency</code> — Telemetry → app render latency (informational; not safety-critical)	ms	150.0	300.0	50.0

Format: 2000 §4.2.5.4 — typical AMS contents. See [../ARCH_DESIGN.md](#) .

Interconnects

AIS — BLE Link

Interconnect: `ai_ble` **Endpoints:** `am_controller`, `am_angler_app` *Generated from `dictionary.yaml`. Do not hand-edit.*

DESCRIPTION

BLE 5.0 GATT service connecting the controller to the angler app. The controller acts as peripheral; the app acts as central. One custom service exposes a telemetry notify characteristic and a config write characteristic.

CARRIES

Architecture flows allocated to this channel:

- `af_telemetry_to_app` — telemetry (data)
- `af_config_to_ctrl` — angler config + override (data)

PROTOCOL STANDARD

Bluetooth Core Specification 5.0. Custom GATT service UUID assigned per project; characteristic UUIDs documented in the app's protocol spec.

DESIGN RATIONALE

BLE chosen over classic Bluetooth or WiFi for low power on the controller side and simple pairing for the angler.

REQUIRED CONSTRAINTS

- **Physical:** Range ≥ 30 m line-of-sight; ≥ 10 m through a vest pocket.

THREAT MODEL (STRIDE)

Per Microsoft SDL (Howard & Lipner 2006). Each row is the narrative describing how the threat category is addressed.

Category	Mitigation
Sp spoofing	Paired-device pairing (BLE LE Secure Connections, P-256 ECDH) prevents unauthorized peripherals from impersonating the controller. The first pairing is in-person (proximity-based); subsequent reconnections use the bonded long-term key.
Tampering	BLE 5.0 LE Secure Connections provides AES-CCM authenticated encryption on every link-layer packet. In-flight tampering would invalidate the MIC and drop the packet.
Repudiation	<code>out_of_scope</code> — single-user session; no third-party audit requirement.
Info disclosure	AES-CCM encryption per BLE 5.0. Telemetry contains fishing-session data only (no PII beyond the angler's pairing key, which is a random bond).

Category	Mitigation
Denial of service	Rate-limit pairing attempts to mitigate pairing-storm DoS. Reconnect backoff (exponential, capped) prevents reconnect-flood. The motor + sensor remain locally authoritative if BLE drops — degraded mode but no safety impact.
Elev. of privilege	Custom GATT service exposes only telemetry-notify and config-write characteristics. No firmware-update path over BLE (firmware updates require physical USB connection — explicit design choice).

Format: 2000 §4.2.6.2 — typical AIS contents. See [../../../../ARCH_DESIGN.md](#).

Modernization

ADRs

ADR — BLE chosen for controller ↔ app transport

ID: `adr_001_ble_transport` **Status:** Accepted **Date:** 2026-05-22 **Author:** controller team

Generated from `dictionary.yaml`. Do not hand-edit.

CONTEXT

The controller needs a wireless link to the angler's mobile device that satisfies three constraints simultaneously: low controller-side power draw (battery-powered enclosure), simple no-account pairing (no cloud login required), and adequate range from a vest pocket. WiFi Direct, classic Bluetooth, and a custom 433 MHz radio were candidates.

DECISION

Use Bluetooth Low Energy 5.0 with a custom GATT service. BLE LE Secure Connections (P-256 ECDH) handles pairing + encryption; paired-device authentication is sufficient for the threat model (single-user session, no audit requirement).

CONSEQUENCES

- Controller-side power budget is met (~10 µA average BLE current).
- Angler app must support BLE on iOS 15+ / Android 11+.
- Range is range-limited to ~30 m line-of-sight, ~10 m through clothing. Acceptable for the angling use case.
- Future expansion to remote-monitoring (cloud forwarding) is independent: the `cloud_forward` bubble uses WiFi/HTTPS, not BLE.

ALTERNATIVES CONSIDERED

- WiFi Direct — rejected: power budget on controller side.
- Classic Bluetooth — rejected: pairing UX worse than BLE; higher power.
- Custom 433 MHz LoRa-style radio — rejected: regulatory complexity, certification cost.

AFFECTS

- **Modules:** `am_controller`, `am_angler_app`
- **Interconnects:** `ai_ble`
- **Flows:** `af_telemetry_to_app`, `af_config_to_ctrl`

CATALOG REFERENCES

MITRE ATT&CK: [T1078](#), [T1565](#)

CWE: [CWE-319](#), [CWE-294](#)

Format: Michael Nygard 2011 — Context / Decision / Consequences / Alternatives. See [../toolkit/MODERNIZATION_DESIGN.md](#).

AutoFishingRig — Service Level Objectives

Generated from `dictionary.yaml`. Do not hand-edit.

SLOs commit external promises about the runtime behavior of the architecture model. Each declares the SLI being measured, the target value, the rolling window, and the error budget that gates release decisions (Google SRE Book 2016).

slo_bite_to_set_latency

Bite detection → **hook-set latency**

- **Target:** 0.2 seconds
- **Window:** 7d
- **Error budget:** 1.0%
- **SLA (customer-facing):** Bite-to-set latency under 200 ms for 99% of detections over any 7-day window.

SLI

p95 latency from tension spike to motor command latch.

```
histogram_quantile(0.95, rate(bite_to_set_seconds_bucket[5m]))
```

Applies to

- **Modules:** `am_controller`

Derived from TPM

[tpm_bite_to_set_currently](#) — Measured bite-to-set latency (lab bench) (current 165.0 ms)

Runbook on budget burn

[runbooks/slo-bite-to-set-burn.md](#)

See [../toolkit/MODERNIZATION_DESIGN.md](#) §4.3 — SLI/SLO/SLA chain.

Runbooks

Runbook — bite-to-set SLO error budget burning

SLO: `slo_bite_to_set_latency` **Trigger:** error budget burn rate above 2× normal over the trailing 1 hour.

SYMPTOMS

- p95 bite-to-set latency rising above 200 ms target.
- Customers may report missed hook-sets ("the rig saw the bite but didn't pull").

DIAGNOSIS

1. **Check tension-sampling rate.** If sample rate has dropped below 100 Hz, the ADC path is the bottleneck — see `tension-sensor-stuck.md`.
2. **Check motor command path.** Latency between `bite_detected` and `motor_command` issued. If high, the bite-detection state machine may be blocked on a fault check.
3. **Check reel motor stall.** If motor isn't responding promptly, mechanical stall in the spool will manifest as bite-to-set latency.
4. **Check enclosure temperature.** ESP32 thermal throttling can slow the bite-detection loop in extreme heat.

RESOLUTION

- **ADC bottleneck:** reduce other sampling tasks if the controller is overloaded; consider firmware update to the 200 Hz ADC path.
- **State-machine block:** inspect for the `state_fault` transition firing too eagerly; tune the fault threshold.
- **Motor stall:** mechanical inspection of the reel spool + drag adjustment.
- **Thermal:** check enclosure ventilation; document customer environment for product team.

ESCALATION

If error budget continues to burn at >2× over 4 hours, page `#oncall-controller-team` and consider pausing new field-beta units until root cause is identified.

Runbook — tension_sensor_stuck

Alert: `tension_sensor_stuck` **Severity:** warning **Module:** `am_controller`
(`proc_acquire_tension`) **Triggered when:** `rate(tension_samples_total[1m]) == 0`

SYMPTOMS

- Tension samples rate drops to zero — no new readings being acquired.
- Bite Detector CSPEC enters `state_fault` after timeout.
- App stops receiving telemetry updates.

DIAGNOSIS

1. Check sensor connectivity (multimeter at load-cell amplifier output — expect 0–3.3V).
2. Check ADC clock + chip-select on the ESP32 via logic analyzer if available.
3. Check firmware logs for `tension.calibration` errors (would indicate cal-table corruption).
4. Verify power rail to the load-cell amplifier (5V $\pm$ 5%).

RESOLUTION

- **Sensor disconnected:** reconnect the load-cell cable; verify in-board screw terminals seated.
- **ADC SPI failure:** power-cycle the controller; if persistent, the ESP32 module needs replacement.
- **Firmware calibration corruption:** restore last known-good calibration from cloud backup or recalibrate from a known mass.
- **Power rail failure:** check enclosure DC-DC converter; replace if output is below spec.

ESCALATION

After 15 minutes without resolution, page `#oncall-firmware` with diagnosis notes. Capture board logs and a photograph of the sensor cable before escalating.

Appendix — HP Quick Reference

Hatley-Pirbhai Quick Reference

Concise refresh on HP vocabulary. Each entry: one-line definition · modern analog where helpful · tiny example · cross-links to related terms.

Anchor links work from chat: e.g. `[DFD](toolkit/reference/HP_QUICK_REF.md#dfd--data-flow-diagram)` . Headings use kebab-case anchors automatically.

Sourced from the 1988 book (*Strategies for Real-Time System Specification*, Hatley & Pirbhai) and the 2000 book (*Process for System Architecture and Requirements Engineering*, Hatley, Hruschka & Pirbhai). Some entries note where they were introduced.

1. The Two (then Three) Models

Requirements Model

The **"what"** half of HP — what the system must do, independent of how. Composed of: a Process Model (DFDs + PSPECs), a Control Model (CFDs + CSPECs), a Requirements Dictionary, and Timing Specs. Modern analog: a behavior spec + state machine + interface contracts, all kept consistent. *See also:* [Architecture Model](#), [Essential vs Implementation Model](#).

Architecture Model

The **"how"** half — physical/logical realization of the requirements. Composed of: Architecture Flow Diagrams (AFDs), Architecture Interconnect Diagrams (AIDs), Architecture Module Specs (AMS), Architecture Interconnect Specs (AIS), Architecture Dictionary. Modern analog: deployment diagram + component diagram + interface contracts + the wiring spec. *See also:* [Architecture Templates](#).

Mechanisms Model (added in 2000)

A **middle layer between Requirements and Architecture**, introduced in the 2000 book. Captures *generic solution patterns* (control loops, state-machine engines, fault-handling schemes) before committing to specific hardware/software modules. Modern analog: design patterns — but named, traceable, and explicit. *See also:* [Layered Models](#), [Enhanced Requirements Model](#).

Essential vs Implementation Model

HP's foundational distinction. **Essential** = what the system must do regardless of technology (the "perfect" technology-independent view). **Implementation** = how it's actually built with real components. The Architecture Model is the bridge. Modern analog: domain model vs deployment model.

Layered Models

The 2000 book's three-tier framing: Essential Requirements Model → Mechanisms Model → Architecture Model. Each layer refines the previous without losing intent. *See also:* [Mechanisms Model](#).

Enhanced Requirements Model

The 2000 book's name for the requirements model *after* the four architecture templates have been applied. Refines the essential model with the bookkeeping the implementation will need (I/O, UI, self-test). *See also:* [The Four Architecture Templates](#).

2. Notation Primitives (what you draw)

Process (Bubble)

The basic data-transformation unit. Drawn as a **circle** with a name and a hierarchical number (e.g., "2.3 Validate Coin"). Each non-leaf bubble decomposes into a child DFD; each leaf bubble has a PSPEC. Modern analog: a function (leaf) or a module (non-leaf).

Terminator

An **external entity** that sits *outside* the system boundary but exchanges data or control with it. Drawn as a **rectangle**. People, other systems, sensors, actuators, regulators. Modern analog: external actor on a context diagram, or "external service" in a service map. *See also:* [Context Diagram](#).

Data Store

A **place data is held** between processes (file, database, table, queue, buffer). Drawn as **two parallel lines** with a name between. Modern analog: a database table, a Redis key, a Kafka topic — anything where one process writes and another later reads.

Control Store

Same shape as a data store, but for **control information** (state values, mode flags). Less common in practice; usually merged with state machines inside CSPECs. *See also:* [CSPEC](#).

Data Flow

A **labeled arrow** from a source (process, store, terminator) to a destination, carrying named data. Solid line. Modern analog: a function argument, a message on a queue, a struct passed somewhere.

Control Flow

Like data flow but carries **events, triggers, or state signals** that *cause* processes to do something. Drawn with a **dashed line** in classic HP. Modern analog: an event, an interrupt, a signal, a webhook.

CSPEC Bar (Short Unlabeled Bar)

A small **horizontal bar** drawn into a DFD that represents a CSPEC reaching into the data world to **activate, deactivate, trigger, or enable** processes. The "magic" notation that connects control to data without cluttering the DFD. Modern analog: the rules engine reaching in to enable/disable handlers based on state. *See also:* [Process Controls](#).

Process Controls

The actual signals the CSPEC bar emits into processes — **activator** (start running), **deactivator** (stop), **trigger** (run once). Modern analog: lifecycle hooks (`start()` , `stop()` , `tick()`).

3. Diagrams

Context Diagram (DCD / CCD / ACD)

The **level-0 diagram** — single bubble (the system) plus all terminators around it, with flows crossing the system boundary. Three variants: **DCD** = Data Context Diagram (data flows only), **CCD** = Control Context Diagram (control flows only), **ACD** = Architecture Context Diagram (the architecture-side equivalent). *Drawn first; the methodology refuses to go deeper until it's locked.* Modern analog: a system context diagram in C4 architecture notation.

DFD — Data Flow Diagram

The **bread and butter** of HP. Bubbles (processes) connected by data flows, with terminators around the edge and data stores between processes. Hierarchical — each non-leaf bubble has a child DFD that decomposes it. Modern analog: a structured flowchart-of-transformations, or a Petri-net-like view of data movement. *See also:* [Leveling](#), [Balancing](#).

CFD — Control Flow Diagram

The **partner to a DFD** for the control-flow dimension. Shows control flows between processes and to/from CSPEC bars. Often drawn on the same sheet as the corresponding DFD. Modern analog: the signaling layer of an event-driven system.

AFD — Architecture Flow Diagram

The **architecture-side counterpart to a DFD**. Shows physical/logical *modules* (rounded rectangles) connected by *architecture flows* (which group the underlying data/control flows). Modern analog: a deployment diagram showing what runs where. *See also:* [AID](#), [Architecture Module](#).

AID — Architecture Interconnect Diagram

Shows the **physical channels** (buses, network links, RS-485, MQTT, file paths) carrying architecture flows between modules. **AID + AFD together = the full architecture picture.** Modern analog: the wiring/network diagram for a deployment. *See also:* [AIS](#).

AICD / AFCD (added in 2000)

Architecture-side **context diagrams** — same idea as DCD/CCD but for the architecture model. AICD = Architecture Interconnect Context Diagram (level-0 of AID). AFCD = Architecture Flow Context Diagram (level-0 of AFD).

4. Specifications (the textual artifacts)

PSPEC — Process Specification

The **leaf-level functional contract** for a bubble that doesn't decompose further. Specifies: inputs consumed, outputs produced, preconditions, postconditions, the transformation rule. Often written in structured English or a decision table. Modern analog: a docstring-plus-contract; a JSDoc/PEP-257 with formal pre/post conditions. *Required for every leaf bubble. See also: [Process \(Bubble\)](#), [Primitive \(Leaf Bubble\)](#).*

CSPEC — Control Specification

The **state/sequencing logic** that controls when processes activate. Usually contains a state-transition diagram, a decision table, or both. Reaches into the DFD via the CSPEC bar to activate/deactivate/trigger processes. Modern analog: a state machine spec, a rules engine config, an FSM library setup. *See also: [STD](#), [Decision Table](#), [CSPEC Bar](#).*

TSPEC — Timing Specification

A **formal capture of timing requirements**: response times, repetition rates, deadlines, latencies. First-class artifact, not just a comment. Example: "F3 (net grid power read) latency ≤ 1 s; sampling rate ≥ 1 Hz." Modern analog: SLO definitions, but at the requirements level. *See also: [Timing Requirements](#).*

AMS — Architecture Module Specification

Like a PSPEC but for an **architecture module** (a deployable component). Describes the module's responsibilities, the data/control flows it handles, its internal allocations. Modern analog: a service README / component contract.

AIS — Architecture Interconnect Specification

Like a PSPEC but for an **interconnect** — a physical/logical channel. Specifies the protocol, format, timing, framing, error handling. **This is HP's answer to "interfaces lost in implementation"** — interfaces are first-class artifacts. Modern analog: a protocol spec, an OpenAPI document, a gRPC IDL, a Modbus register map.

Requirements Dictionary

The **central glossary of every named flow, store, and data element** in the requirements model. Every arrow/store/element on every diagram has an entry. Modern analog: a data dictionary, schema registry, or shared types library. *See also: [Group Structure](#), [Primitive Attributes](#).*

Architecture Dictionary

The dictionary's architecture-side counterpart — every architecture module, flow, and interconnect has an entry.

Group Structure

A way of specifying **composite data elements** in the Requirements Dictionary (e.g., "telemetry_packet = timestamp + channel_id + voltage + current"). Modern analog: a struct/record type definition.

Primitive Attributes

The leaf-level attributes in the dictionary — units, ranges, precision, encoding. Modern analog: a primitive type definition with constraints.

5. Concepts & Discipline

Leveling

The **hierarchical decomposition** of DFDs. Level-0 = context diagram (one bubble). Level-1 = the first decomposition (3-7 bubbles). Each non-leaf bubble at level N becomes its own DFD at level N+1. The leveling rule: **every bubble at the same level is at the same level of abstraction**. *Anti-rathole mechanism: you can't go deep on one bubble while leaving siblings under-specified.*

Balancing

The conservation law of DFDs. **Flows in and out of a parent bubble must match flows in and out of its child DFD**. A child DFD must preserve the parent's interface exactly. *Modern analog:* a function's signature must be consistent with what its body actually consumes and produces. Validation: balancing checks find drift early.

Decomposition

The process of taking one bubble and breaking it into a child DFD with multiple bubbles. HP demands that decomposition be *justified* — typically because the bubble has 3+ distinct responsibilities, or its PSPEC would be too long. *See also:* [Leveling](#).

Seven-Plus-or-Minus-Two

HP's rule of thumb: **a DFD should have 5-9 bubbles**. Below 5, you haven't decomposed enough. Above 9, the diagram is incomprehensible. Sourced from Miller's classic cognitive-load paper. Modern analog: the "team size of two pizzas" or "module file under 500 lines" heuristic.

Traceability

The **explicit links** from requirements to architecture to implementation (and ideally to tests). HP demands that every requirement be allocated to an architecture module, and every architecture module justify itself against requirements. Modern analog: requirement-IDs on commits, story-to-test mapping, but enforced as a first-class artifact. *See also:* [Traceability Matrix](#), [Extended Traceability](#).

Traceability Matrix

The **table** that records traceability links — typically requirements as rows, architecture modules as columns, with cells marking allocations. *Anti-85%-syndrome mechanism: uncovered requirements stand out as empty rows.*

Primitive (Leaf Bubble)

A bubble that **doesn't decompose further** — its behavior is captured in a PSPEC instead of a child DFD. The bottom of the hierarchy. Modern analog: a pure function with a docstring.

System Hierarchy / Universal Hierarchy

HP's view that all systems exist in a containing hierarchy — your system is part of a larger system (the *environment*), and contains smaller systems (the *subsystems*). The context diagram defines the boundary between *your* system and the level above it. *See also:* [Multiple Hierarchies](#).

Transaction Center

A bubble that **dispatches between multiple downstream processes based on input** (often command/event-driven). Modern analog: a switch statement, a router, a dispatcher.

Primitive Network

The DFD at its **finest level of decomposition** — only leaf bubbles remain, all with PSPECs, no further decomposition possible. Modern analog: the "leaf functions" of a refactored codebase.

6. The Four Architecture Templates

The 1988 book's contribution: every architecture wraps the requirements model in **four standard "process blocks" surrounding a core**. These are the things real systems need that pure structured analysis ignored.

Input Processing

The **block that handles incoming data/control from terminators** — sensor reads, network input, button presses, file reads. Decouples the rest of the system from physical-input concerns. Modern analog: input adapters in hexagonal architecture.

Output Processing

The **block that handles outgoing data/control to terminators** — actuator commands, displays, network output, file writes. Modern analog: output adapters in hexagonal architecture.

User Interface Processing

The **block that handles human interaction** — dashboards, configuration, manual override, alerts. Separate from generic I/O because human latency, error tolerance, and presentation needs differ. Modern analog: presentation layer.

Maintenance / Self-Test / Redundancy Management

The **block that handles non-functional concerns** — health checks, diagnostics, failover, recovery, redundancy. The "what keeps this thing running" layer. Modern analog: SRE/observability + HA features.

Architecture Template (4 surrounding blocks)

The collective name. Every architecture is the **requirements model "core" surrounded by these four blocks**. Forces you to think about I/O, UI, and self-test as first-class architecture concerns, not afterthoughts.

7. Control Formalisms (inside CSPECs)

FSM — Finite State Machine

Classic state machine: states + transitions + events + actions. HP uses FSMs inside CSPECs for any process with non-trivial sequencing. *See also:* [STD](#), [State Transition Matrix](#).

STD — State Transition Diagram

The **graphical form of a state machine** — circles for states, arrows for transitions labeled with event/action pairs. Modern analog: a state-chart in XState or similar.

State Transition Matrix / Table

The **tabular form of a state machine** — rows are current states, columns are events, cells are next-state-and-action. Equivalent to STD, sometimes more compact for large state spaces.

Decision Table

A **tabular form for combinational logic** (no state). Rows are conditions, columns are rule combinations, cells say which actions fire. Modern analog: a rules engine ruleset, a truth table.

Combinational vs Sequential Machine

Combinational = output depends only on current inputs (decision table). **Sequential** = output depends on inputs *and* current state (STD/state matrix). HP CSPECs may contain either or both.

State Chart (Harel, added in 2000)

A **hierarchical state machine** with nested states and parallel regions. From David Harel's 1987 work. The 2000 book brings these into HP for cases where a flat FSM is too noisy.

Composite CSPEC / Multi-Sheet CSPEC

A **CSPEC that spans multiple sheets** because the control logic is too complex for one page. Same logic, just paginated.

8. 2000-Book Additions

The 2000 book (Hatley/Hruschka/Pirbhai, "PSARE") added a *process* layer on top of the 1988 *notation*, plus refinements.

Process / Methods / Tools Precedence

The 2000 book's mantra: **process drives methods, methods drive tools** — pick in that order, not the reverse. Modern analog: "don't pick the tool first."

Concurrent Development Process

The 2000 book's alternative to waterfall: requirements, mechanisms, and architecture evolve *concurrently*, with iteration loops between them. Modern analog: agile-but-with-actual-architecture-work.

Superbubble

A **virtual bubble grouping multiple related bubbles** for navigation or abstraction purposes. Doesn't change the leveling rules; just a viewing aid. Modern analog: a "lens" or "selection" in a diagram tool.

Extended Traceability (added in 2000)

Refinement of 1988's traceability: links extend not just requirements→architecture but **requirements→mechanisms→architecture→implementation→tests**, with bidirectional navigation. See also: [Traceability](#).

Derived Requirement

A **requirement that emerges during architecture or design** — not in the original stakeholder ask, but necessary because of how the system is being built. The 2000 book makes these first-class so they're traceable too. Modern analog: "non-functional requirement discovered during implementation."

Object Orientation Integration

The 2000 book acknowledges UML had arrived and offers patterns for combining HP's structured analysis with OO design (class diagrams alongside DFDs, state charts in CSPECs). Modern analog: "structured analysis and OO are complementary, not exclusive."

Multiple Hierarchies (added in 2000)

Recognition that real systems live in **multiple overlapping hierarchies** (organizational, physical, functional, control). The 1988 book assumed one. The 2000 book lets you model several.

Meta-Model / Development Process Meta-Model

The 2000 book's framing of the *development process itself* using the same notation. The development project becomes a system, the team are processes, the deliverables are flows. Reflexive use of HP on the work of doing HP. Modern analog: "we used our own tool on our own project."

Business Object Architecture

A 2000-book pattern for organizing architecture around **business-domain objects** rather than purely-functional decomposition. Bridge between HP and domain-driven design.

Workflow-Centered Architecture

2000-book pattern: organize the architecture around **end-to-end workflows** (start at the user, follow the data, end at the persistence layer). Modern analog: vertical slice architecture.

9. Common Examples (from the 1988 book)

The HP books illustrate everything through four canonical worked examples, in increasing complexity. Worth knowing about as reference points:

- **Vending Machine** — transactional, discrete events, basic FSM. The "hello world" of HP.
 - **Home Heating System** — continuous control with sensors and actuators. Introduces continuous data flow and closed-loop control.
 - **Automobile Management** — multiple cooperating subsystems. Introduces inter-process control and decomposition across subsystems.
 - **Flight Management (Avionics)** — full real-time, safety-critical, redundant. The "go-to" complex example; the methodology's origin domain.
-

10. Antecedents & Influences

HP didn't appear in a vacuum — it builds on (and credits) several prior methods:

- **DeMarco Structured Analysis** (1979) — the original DFD notation, no control.
 - **Yourdon/Constantine Structured Design** (1979) — the architecture-side counterpart, structure charts.
 - **Ward-Mellor Real-Time Structured Analysis** (early 1980s) — an alternative real-time extension to DeMarco. HP and Ward-Mellor are parallel approaches with different control notations.
 - **Finite State Machine Theory** — the formal underpinning of CSPECs.
 - **Harel State Charts** (1987) — incorporated into the 2000 book.
 - **Putnam Model** — project sizing/scaling, referenced in the 1988 book.
-

This is a quick-reference card, not a tutorial. For the deep dive, see the source books: Hatley & Pirbhai (1988), Strategies for Real-Time System Specification; and Hatley, Hruschka & Pirbhai (2000), Process for System Architecture and Requirements Engineering.

Last updated: 2026-05-22.